

Jack 3.8 27b Coder in Long-Context Agentic Work

How Jack XML filtered and carried operative state forward across a long software project on a 16 GB GPU

Updated longitudinal analysis through C14

Large language models are very good at solving individual problems. Give one a programming task, a bug, or a design question, and a capable model can often produce an impressive answer.

Long-running software work is harder.

A real project may require dozens of decisions spread across many hours of work. Files change. Tests reveal problems. Earlier plans become outdated. A model has to remember which parts are finished, which conclusions were rejected, which interfaces must not change, what the latest test proved, and what it should work on next.

The problem is not simply having a large context window.

The real problem is **keeping the model aligned with the current state of the project**.

That was the purpose of this experiment with **Jack 3.8 27b Coder**.

The experiment was not intended to prove that the underlying 27B model was exceptionally powerful. Jack 3.8 was deliberately designed to fit and operate on a **16 GB GPU**. The more interesting question was whether a model in that practical hardware class could use **Jack XML** to carry important information forward and remain organized during a long agentic software-development task.

The test project was a browser recreation of Atari's 1980 game *Battlezone*. It included vector graphics, tank controls, projectiles, radar, enemies, collision detection, procedural sound, scoring, a state machine, browser integration, tests, verification logs, and persistent project records.

The important part was not the game itself.

The important part was how Jack moved through the build.

This distinction matters for interpreting the experiment. **The primary object being evaluated is not the quality of every line of JavaScript Jack produced.** Coding output matters only as secondary evidence that the carried state remained useful. The primary question is whether Jack XML could repeatedly filter a growing project history into a smaller operative state: preserving what still mattered, demoting what had become historical, carrying unresolved uncertainty, and rebinding the project when new evidence or a new user instruction changed what was current.

The problem Jack XML is trying to solve

Imagine working on a software project with another person.

At first, remembering everything is easy. You both know what was just discussed.

After several days, the situation becomes different. There may be an old plan, a newer plan, several repaired bugs, a changed function interface, three failed experiments, a test that was rewritten, and a decision that must never be reversed.

All of that information still exists somewhere.

But not all of it is equally important anymore.

Language models face the same problem.

A long conversation can contain:

- old plans,
- current plans,
- obsolete assumptions,
- new test results,
- code that has since changed,
- previous failures,
- current file identities,
- decisions that must remain fixed,
- and unresolved problems.

A large context window allows the model to *see* more of this history.

That does not automatically tell the model **which pieces represent the current truth**.

Jack XML attempts to solve this by repeatedly converting project history into a smaller, structured representation of the project's **operative state**.

Instead of carrying the entire project forward as prose, Jack records what matters now.

That includes things such as:

- the current verified baseline,
- the requirement currently being worked on,
- requirements already verified,
- exact file hashes,
- important function interfaces,
- fixed numerical constants,
- unresolved problems,
- relevant decisions,
- what files need to be inspected next,
- and a specific failure condition Jack should attempt to falsify.

By C5 of the Battlezone build, for example, Jack's checkpoint recorded that input handling had been verified, radar was the next objective, exact hashes existed for the current math, renderer, and input files, and several other modules were still empty. It also preserved exact values such as the 800x600 canvas, 400 FOV, 0.2 near clip, 12-unit maximum speed, 1.8 rad/s yaw rate, and the radar's position and range.

The XML was therefore doing something more useful than saying:

"Input is finished. Next, work on radar."

It was carrying forward enough structured state to help the next round of reasoning begin from the actual project rather than from a vague memory of it.

The seven-stage Jack loop

The Battlezone experiment used a seven-stage development loop:

PLAN -> EXECUTE -> REVIEW -> AUDIT -> REPAIR -> VERIFY -> RECONCILE

Then Jack returned to PLAN and began the next bounded piece of work.

The stages sound simple, but the separation between them is important.

1. PLAN

Jack first examines the live project and asks what the governing specification actually requires.

It chooses a bounded next task instead of trying to design the entire future project in detail.

The plan identifies the files, interfaces, dependencies, risks, and tests that matter for that step.

Then Jack XML freezes that operative state.

2. EXECUTE

The outer coding agent performs the actual work.

Files are changed. Code is written. Tools are run.

Jack does not treat generated code as completed work just because the model produced it. The project's instructions distinguish between code that was merely proposed, code that was actually written to disk, and code whose behavior was verified.

3. REVIEW

Review asks a relatively simple question:

Did we actually implement what we planned?

This catches missing files, accidental interface changes, wrong names, incomplete work, and differences between the plan and the resulting code.

4. AUDIT

Audit asks a harder question:

Even if we implemented the plan correctly, how could the result still be wrong?

This distinction became very important later in the experiment.

The audit looks for incorrect assumptions, test weaknesses, state-transition errors, cross-file inconsistencies, and other failures that a simple comparison against the plan may miss.

5. REPAIR

If Review or Audit finds a real defect, Jack makes the narrowest justified repair.

The idea is not to rewrite everything every time something goes wrong.

Already-correct work should remain intact.

6. VERIFY

Now the system asks whether the resulting implementation actually works.

That means deterministic evidence: browser tests, Node tests, file inspection, hashes, runtime behavior, or other checks appropriate to the requirement.

7. RECONCILE

Finally, Jack updates the project's trusted state.

The new checkpoint records what is now verified, which files changed, what interfaces remain binding, which decisions are active, what evidence supports the result, and what should be read next.

Only after reconciliation does the next PLAN begin.

This last stage is the key to long-context work.

Without it, each new task starts from a mixture of old conversation history and whatever the model happens to remember.

With it, Jack repeatedly establishes:

This is where the project is now.

Filtering, not merely remembering

One of the strongest results of the Battlezone experiment was not simple persistence. It was **selective state filtration**.

A weak memory system can keep adding facts forever. A useful long-horizon state system has to do something harder: decide what remains operative now.

The checkpoint sequence shows that happening repeatedly.

At C6, projectile ballistics was the active frontier. Jack carried the already-verified input and radar work as background state, preserved the exact binding interfaces and frozen constants that ballistics depended on, marked the still-missing World integration explicitly, and focused the next pitfall_check on strict single-shot behavior, rising-edge firing, inherited momentum, TTL, range, obstacle termination, and enemy-shell rules.

At C7, ballistics stopped being the active uncertainty. It became a **verified dependency**. The active frontier moved to World geometry and collision. Jack kept the ballistics interface because World now had to integrate with it, but replaced the ballistics pitfall with a World-specific one: deterministic generation, exact obstacle counts, spawn clearance, collision boundaries, and real integration with shell collision.

At C8, World was likewise demoted from active problem to trusted background state. The frontier moved to enemies. Jack retained only the World and projectile interfaces that the enemy phase needed while replacing the active falsification target with enemy geometry, kinematics, per-enemy shells, scoring, and collision behavior.

At C9, enemy behavior, spawn configuration, LOS rules, scoring, map facts, and model interfaces had been compressed into the operative state while audio became the next active requirement.

At C10, the earlier systems were represented as verified dependencies and the active frontier had narrowed to the master game loop.

This pattern can be summarized as:

active uncertainty -> verified dependency -> compressed interface or invariant -> retained only when later work still depends on it

That is different from merely remembering previous conversation turns.

Jack XML was repeatedly changing the **priority structure** of project information.

The model did not need to reconstruct the entire project from scratch at each stage. At the same time, the XML was not allowed to replace the repository as source of truth. The live project remained authoritative.

The XML served as a map of the operative state: what was trusted, what remained uncertain, what had changed, and what mattered next.

This is why the experiment is better described as a test of **filtering and carry-forward** rather than raw memory.

The C6 regression showed experience compression

One of the more revealing events occurred during C6.

A test file, `run_node.mjs`, silently returned to an earlier version. Its newly added radar tests disappeared.

A weaker workflow could have interpreted a clean-looking test run as success. Jack instead noticed that an expected radar test section was missing from fresh output. The checkpoint recorded the incident in detail: the file had regressed to its C5 identity, the missing section was restored, and a new rule was established requiring expected markers to be checked after multi-file edits.

The important result is what happened **after** the incident.

At first, the event was preserved in relatively detailed form because it was recent and operationally important.

By later checkpoints, Jack no longer carried the whole story. The incident had been compressed into a smaller durable rule:

after a multi-file edit, verify that the expected markers actually exist; missing expected output is evidence of state regression, not evidence of success.

As additional testing lessons accumulated, that rule itself evolved. Later checkpoints added the discipline of using numerically verified corridors instead of hand-computed geometry and of trusting fresh tool output over stale assumptions.

The information trajectory therefore looked like:

specific failure -> causal lesson -> compact reusable operating rule

That is more interesting than ordinary note taking.

Jack XML was not simply storing an execution diary. It was preserving the **consequence** of the event while allowing the detailed event history to become less prominent.

This is one of the clearest examples in the experiment of long-context information being **distilled rather than merely retained**.

Jack XML also carried uncertainty forward

Good state management is not only about remembering what is complete.

It also requires remembering what is **not yet proven**.

C7 is a good example.

Ballistics had been implemented before the complete World module existed.

Instead of pretending that obstacle interaction had been fully proven, the checkpoint distinguished between projectile behavior that could already be tested and integration behavior that needed to wait for the real World implementation.

C8 then replaced that temporary assumption with the actual World implementation and verified collision through the real path.

That is exactly the kind of distinction that becomes easy to lose in a very long conversation.

A model may remember that "ballistics passed," but forget that one part of that result depended on a stub.

Jack XML gave the system a place to preserve that difference.

The pitfall check

Each major checkpoint also contained a `pitfall_check`.

The idea was to force Jack to identify a **falsifiable crux**: one of the most important ways the next step could be wrong.

For radar, Jack focused on correct yaw-relative positioning, range clamping, and directional mapping.

For ballistics, it focused on rising-edge firing, strict single-projectile behavior, velocity inheritance, TTL, range, obstacle impacts, and enemy-shell rules.

For World, it focused on exact obstacle counts, deterministic generation, spawn clearance, collision boundaries, and integration with shell collision.

For audio, Jack explicitly carried the required frequencies, durations, gains, filters, and initialization behavior into the next verification step.

This is a useful feature of Jack XML because it moves the model away from a vague instruction such as:

"Check your work."

Instead, the model asks:

What evidence would most strongly prove that my current understanding is wrong?

That makes the structured state active rather than passive.

The anchor set evolved with the project

The checkpoint chain also shows that the XML anchor set was not static and did not simply grow without control.

Across the early checkpoints, a small collection of project invariants remained continuously active: viewport dimensions, FOV, near clip, visual constants, movement limits, radar parameters, shell limits, world counts, scoring, and state-machine names.

New anchors appeared when new information became durably relevant.

When enemy spawning became part of the live project, the state gained a compact spawn-configuration anchor. When audio became operative, the state gained an engine-audio anchor. When the user later changed projectile appearance, C13 added a SHELL_BALL anchor while preserving the older shell-physics anchor underneath it. The representation therefore distinguished **how the projectile looked** from **how the projectile behaved**.

When the user later asked for fewer tanks, C14 did not rebuild the whole spawn representation. It updated the relevant spawn state with TANK_CAP = 2 while preserving the other spawn intervals, score thresholds, saucer timing, physics, and scoring rules.

This suggests a useful property of the XML state:

it was mutable, but conservative.

Stable invariants remained stable. New durable information was promoted when needed. User-authorized changes modified the relevant part of the state without forcing unrelated information to be reinterpreted.

That is the behavior expected from a working operative-state representation rather than an ever-growing summary.

C11: when a consistent project state was still wrong

The most valuable lesson came near the end.

By C11, Jack had assembled the major systems into a Game class.

The Node tests passed.

The browser integration test ran.

The game scored a kill.

The canvas contained graphics.

The persistent records agreed.

Jack declared the build complete.

There was only one major problem.

The actual game did not launch from the real production page.

The integration test had created the Game object itself and started it.

The shipped index.html loaded main.js, but main.js did not actually create and start the game.

In other words, the test proved that the internal game object could work.

It did not prove that the application as delivered could start.

This was a very useful failure for the experiment.

C11's final audit had become focused heavily on whether Jack's persistent state files agreed with one another.

They did.

But agreement between records is not the same thing as agreement with reality.

That is an important lesson for any long-context agent architecture:

A perfectly consistent memory can still consistently remember the wrong conclusion.

C12: Jack challenged its own completed state

The next event is probably the strongest result of the entire experiment.

Jack was not told:

"You forgot to instantiate the Game class."

It was only told that the game was not working and that it should debug the project through the seven-stage loop.

That gave the system a symptom rather than the answer.

C12 reports that Jack reproduced the problem through the actual production endpoint. It observed a black canvas and no active Battlezone runtime. It then traced the problem to the missing production bootstrap and recognized that the earlier integration test had hidden the failure by constructing its own game instance.

That is exactly the kind of behavior Jack XML is meant to support.

The prior state said:

COMPLETE.

Reality said:

The game does not work.

Reality won.

Jack reopened the problem.

It repaired the production startup path.

It also changed the verification strategy so that the same mistake would be harder to repeat.

The new integration test was redirected through the real shipped entrypoint rather than constructing its own game.

Jack also added a browser-debugging probe that used real Chrome keyboard events instead of an internal simulated-input function. The C12 state records testing of actual startup, attract-mode rendering, Space transitioning into play, WASD movement, projectile behavior, canvas output, and browser errors.

This is an important difference.

C11 tested:

Can the Game class run?

C12 tested:

Does the shipped application actually start and respond through the same path a user will use?

That is a much stronger test.

What C12 says about state rebinding and compaction

C12 did not make Jack perfect, but its significance is clearer when the experiment is judged as an operative-state test rather than as a coding benchmark.

The prior state said:

COMPLETE.

New external evidence said:

The shipped application does not work.

Jack did not preserve the earlier conclusion merely because it had already been reconciled. It rebuilt the operative state around the new evidence, carried forward the useful earlier interfaces and invariants, replaced the production-entry assumption, and promoted a new persistent testing rule: production verification must exercise the actual shipped entry path rather than a privately constructed component path.

That is **state rebinding**.

A second issue in C12 came from a different layer of the system. The C12 state contains two different identities for src/radar.js. In this experiment that duplicate was introduced by **host-side context compaction**, not by Jack's own XML filtering process.

That distinction is important.

Jack XML and the host's context-compaction mechanism are not the same thing. A lossy compaction layer can contaminate the information presented back to an agent even when the agent's explicit state format is structured.

Rather than treating the duplicate radar identity as evidence that Jack XML itself failed to filter state, it is better understood as evidence for a separate long-context hazard:

external context compaction can inject corruption into an otherwise explicit operative state.

The explicit checkpoint format makes that corruption visible. It creates a state object that can be compared before and after a compaction boundary instead of silently trusting whatever compressed context the host reconstructs.

The subsequent checkpoint is also informative. By C13, the manifest had returned to a single coherent radar identity while the useful C12 lessons - real production entry, real keyboard probing, and the masked-by-bypassed-entry failure mode - remained in the operative state.

So the important C12 result is not that Jack XML guarantees perfect state.

It is that the state is **explicit enough to be challenged, replaced, inspected for contamination, and carried forward after correction.**

C13: filtering debugging noise into probe discipline

C13 added another kind of evidence for filtration.

The user asked for fired projectiles to be displayed as glowing balls. During verification, several apparent failures occurred. The important point is not the projectile rendering itself. The important point is how the resulting state handled the debugging noise.

The final C13 checkpoint records that the misleading failures came from probe-side causes: invalid Chrome DevTools keyboard values, instrumentation consuming the fire edge before the game step, sampling after the useful projectile window, and uncontrolled scene conditions.

Jack did not carry forward the entire exploratory debugging transcript. It extracted the durable consequences.

The checkpoint preserved the exact projectile representation and the fact that the underlying shell physics must remain unchanged, while also turning the failed probe experience into permanent testing discipline.

By C14, even those details had been compressed further into a concise operational rule about valid CDP key values, non-disruptive telemetry, in-flight sampling, and never consuming fireEdge from instrumentation.

Again the information trajectory was:

noisy history -> causal extraction -> compact operative knowledge

This is exactly the kind of transformation a long-horizon agent needs. Most failed attempts should not remain equally salient forever. Their useful lessons should.

C14: narrow state mutation after completion

C14 tested a different property of the state system.

The project had already reached a reconciled baseline when the user supplied a new preference: there were too many tanks on screen.

Jack did not reopen the entire project or reinterpret every spawn rule. The state identified the narrow lever that could satisfy the request without disturbing the surrounding contract: the simultaneous tank cap.

The operative state changed from a cap of four tanks to `TANK_CAP = 2` while explicitly preserving the spawn interval, score thresholds, saucer interval, enemy type behavior, ballistics, scoring, and other unrelated invariants.

This can be represented as:

trusted state + authorized delta -> revised trusted state

That is a particularly clean example of selective state mutation.

A useful long-running agent should not merely remember old decisions. It should also know which parts of the current state are allowed to change when a new instruction arrives, and which surrounding commitments must remain untouched.

C14 provides direct evidence of that behavior.

Why the 16 GB GPU target matters

Jack 3.8 27b Coder was designed to be usable on a **16 GB GPU**.

That makes this experiment interesting for a practical reason.

A common approach to agentic AI is to solve problems with more of everything:

larger models, larger context windows, more inference, more memory, and more hardware.

Jack explores another direction.

Instead of expecting the model to keep an entire project perfectly active inside its neural context, the surrounding architecture divides responsibility:

- source code stays in the repository,
- deterministic tools establish evidence,
- logs preserve detailed test output,
- ledgers preserve durable decisions,
- and Jack XML carries the current operative state.

The model therefore does not need to *be* the entire project memory.

It needs to know:

What is currently true?

What has been verified?

What is still uncertain?

What should I inspect next?

What must not change?

What evidence would prove me wrong?

That is a much more efficient way to think about long-horizon local agents.

The point of the 16 GB constraint is therefore not that Battlezone requires powerful hardware. It obviously does not.

The point is that a 27B-class local model operating within a practical VRAM budget was able to participate in a lengthy, multi-stage engineering process while a structured external state system helped keep that process coherent.

That is the architectural result.

The 27B model should therefore be understood as the **test substrate**, not as an architectural limit of Jack XML. The same filtering, checkpointing, and state-rebinding approach could sit above a much larger model. A larger reasoning engine might improve planning or auditing, but the long-context problem would remain: it would still need a reliable way to distinguish current operative state from accumulated history.

Jack XML as a rolling operative-state filter

The Battlezone build suggests a more precise way to think about Jack XML.

It is not merely a format for model output, and it is not just a conventional project summary.

It acts as a **rolling operative-state filter between historical context and the next action**.

Across the checkpoint chain it performs three related jobs:

- **Selection:** What from the past still matters now?
- **Compression:** At what level of detail should that information survive?
- **Rebinding:** What becomes the current operative state after new evidence or a new user instruction?

After each important project transition, Jack can re-establish:

This is the verified baseline.

These are the interfaces that now exist.

These constants remain binding.

These requirements are finished.

These remain unresolved.

This is the next smallest gap.

This is the strongest assumption we should try to falsify.

The process then repeats.

In simplified form:

PLAN

v

Freeze current operative state

EXECUTE

v

Change the real project

REVIEW

v

Compare intended work against actual work

AUDIT

v

Try to discover how the apparently correct result might still fail

REPAIR

v

Correct demonstrated defects

VERIFY

v

Collect deterministic evidence

RECONCILE

v

Build the next trusted operative state

PLAN AGAIN FROM REALITY

That final sentence is perhaps the most important part.

Plan again from reality.

Not from the original roadmap.

Not from an old conversation.

Not from what the model vaguely remembers.

Not even from the previous XML if the live system contradicts it.

Reality remains the authority.

Jack XML helps the model stay oriented toward it.

Conclusion

The Battlezone experiment was not a benchmark of raw model intelligence or a scorecard for every line of generated code.

It was a test of whether **Jack XML could help a local model filter and maintain operative state across long-context agentic work.**

The checkpoint chain from C5 through C14 provides encouraging evidence.

Jack repeatedly preserved exact project invariants while changing the active frontier. Requirements moved from active uncertainty into verified dependency state. Interfaces survived when later work still depended on them. Temporary assumptions remained marked as temporary until real dependencies existed. New durable information was promoted into anchors only when it became relevant.

The system also showed a more interesting kind of compression.

A real C6 regression began as a detailed incident and later survived mainly as a compact procedural rule. C13's noisy probe failures were reduced to durable instrumentation discipline rather than being carried forward as an execution diary. C14 incorporated a new user instruction as a narrow state delta while preserving unrelated commitments.

The C11-to-C12 transition showed that the operative state could be replaced when reality contradicted an earlier completion claim. The previous COMPLETE state did not become untouchable history. New evidence reopened the project, and the consequences of the discovery were promoted into the next trusted baseline.

C12 also exposed a separate long-context problem outside Jack XML itself: host-side context compaction can contaminate the state presented back to an agent. Because the checkpoint is explicit, that contamination can be observed and compared rather than remaining hidden inside an opaque summary.

The important achievement is therefore not that XML somehow made a 27B model smarter.

The more interesting possibility is that structured operative state helped the model **use the intelligence it already had across a much longer span of work.**

The 16 GB constraint makes the demonstration practical, but it does not define the architecture. Jack XML is a control and state-maintenance approach that could in principle sit above much larger models as well.

The deeper result is that long-context work is not mainly a challenge of remembering everything.

It is a challenge of **forgetting intelligently:**

- preserving what remains operative,
- compressing what has become historical,
- retaining uncertainty until it can be resolved,
- converting failures into reusable rules,
- and replacing yesterday's state when today's evidence changes reality.

A model does not need every previous thought to remain equally active.

It needs a reliable answer to a smaller set of questions:

Where are we now?

What do we actually know?

What is still uncertain?

What must remain fixed?

What changed?

What should happen next?

The C5-to-C14 Battlezone checkpoint sequence suggests that Jack XML can provide a surprisingly strong answer to those questions over a long agentic trajectory.